

AER1217 Lab 3

Dejan Plavsic - 1012836442

Chris McGale - 1005303841

Mohsen Movahedi - 1005371517

1.0 Purpose

The purpose of this lab was to locate and estimate the position of six targets from images captured on a camera mounted to the bottom of a drone in flight. In addition to the 2000+ images, the position of the drone was provided via a Vicon system, giving position and orientation.

2.0 Derivations

Given the position and orientation of the drone it was possible to determine the positioning of the detected targets via the visual odometry pinhole model in the following procedure. The first step was to correct the displayed pixel value for distortion via the cv2.undistortPoints function and the provided d array for the given camera system used.

$$d = [0.191887, -0.563680, -0.003676, -0.002037, 0.000000]$$

After the lens distortion was accounted for it was possible to calculate the ray (line) from the camera sensor to the target using information regarding the focal length and principal point via the pinhole model. The calculation can be expressed in the following form where K is a matrix containing information regarding image depth.

$$[x_{ray}, y_{ray}, z_{ray}] (local) = K^{-1} [x_{pixel}, y_{pixel}, 1]$$

This ray is in the local camera frame convention however and need to be converted to the global Vicon sign convention. This requires first converting the camera frame into the drone (body) frame of reference, and then converting this into the global Vicon frame of reference. The transformation matrix for the camera is provided in the appendix, and is just a flip of the axis (Tcb). The transformation matrix for the body to global frame of reference (Tbg) requires the use of quaternions, as the orientation of the drone (body) is provided in this sign convention. This convention has the advantage of avoiding 'gimblelock' which is experienced when using Euler angles. The Scipy Rotation library was used to convert the quaternion angle convention into a transformation matrix.

$$[x_{ray}, y_{ray}, z_{ray}] (global) = T_{GB} T_{CB} [x_{ray}, y_{ray}, z_{ray}] (local)$$

With the global ray position and orientation known, it then required finding the intersect of ray with the target to determine the exact point in space of the target. Using the assumption that the target is on the x-y plane (z=0), it was possible to determine a scalar of the ray, based on the assumption that the pz position of the drone, must be equal to the z component of the scaled ray. Knowing the position of the drone (body), and the vector to the target it was hence possible to determine the position of the target in the global frame of reference.

$$\lambda = -p_z / z_{ray}$$
$$(x, y, z) (target\ position) = [p_x, p_y, p_z] + \lambda [x_{ray}, y_{ray}, z_{ray}] (global)$$

3.0 Implementation:

3.1 Target Detection

The features present in the target that were used in our detection are the following:

- The contrast between the white rectangle of the target and the grey floor – detected by thresholding
- The rectangular shape of the target background – detected using contours and BoundingRect
- The circular target within the white rect – detected using HoughCircles

The overall strategy for circle target detection via rectangle ROI and hough transform:

Finding the regions of interest (ROI):

- Apply a high binary threshold to only detect the white rectangles
- Denoise the shapes by applying a Rectangular morphological mask
- Find the contours of the binarized image
 - Find a rectangle by using the cv2.AproxPolyDP
 - If poly is a rect, find the BoundingRect of said poly
 - Save the pixel coordinates of the ROI
- Finding the target circle:
 - Isolate the ROI on the Grayscale slightly blurred image
 - Approximate the largest circle using cv2.HoughCircles
 - Take the most confident circle as the target and save its original pixel coordinate

FUNCTION DetectTarget(img):

```
1: gray ← Grayscale(img)
2: gray ← MedianBlur(gray, 5)
3: binary ← Threshold(gray, min=127, max=255, cv2.THRESH_BINARY)
4: binary ← MorphClose(binary, 3×3 Rect_kernel)           # Give definition to rects
5: binary ← MorphOpen(binary, 3×3 Rect_kernel)
6: contours ← FindContours(binary)
7: FOR each contour c ∈ contours DO
8:   approx ← cv2.AproxPolyDP(c, ε = 0.02·Perimeter(c))
9:   IF Vertices(approx) = 4 THEN                             # Find the Rectangle
10:    (x, y, w, h) ← cv2.BoundingRect(c)
11:    IF 25 < w < 320 AND 25 < h < 180 THEN
12:      roi ← Crop(gray, x, y, w, h)                         # Map ROI to gray image
13:      circles ← HoughCircles(roi)                          # Approximate a circle
14:      IF circles detected THEN
15:        circle ← First(circles)
16:        cx ← circle.x + x
17:        cy ← circle.y + y
18:        r ← circle.radius
19:        RETURN {center:(cx,cy), radius:r}                  # Return circle parameters
24: RETURN None
```

3.2 Localization

For every target detected using the process in section 3.1, we save the pixel coordinates of the center of the circle, its radius and the frame it is in are recorded. In order to find the world coordinates of the targets, each circle detection goes through the Georeferencing pipeline described in the Derivations section:

- Given the pixel coordinates (u,v):
 - Undistort the pixel using `cv2.undistortPoints`
 - Get coordinates in Camera frame from pixels, using intrinsic matrix K
 - Get coordinates in Body frame from Camera frame, using the transformation matrix (T_{CB})
 - Finally, get the world (Vicon) frame coordinates, using the rotation matrix (R_{WB}) extracted from the Vicon pose quaternions for that particular frame

FUNCTION PixelToWorld(pixel, position, R_WB, K, d, T_CB):

```
1: points ← Format pixel as (1×1×2)
2: pixel_u ← cv2.UndistortPoints(points, K, d, P = K)           # undistort pixel
3: p_homogeneous ← (pixel_u.x, pixel_u.y, 1)
4: ray_cam ←  $K^{-1} \cdot p\_homogeneous$                        # pixel to camera frame
5: ray_cam_homogeneous ← (ray_cam.x, ray_cam.y, ray_cam.z, 0)
6: ray_body_h ←  $T_{CB}^{-1} \cdot ray\_cam\_homogeneous$          # camera to body frame
7: ray_body ← ray_body_h[:3]
8: ray_world ←  $R_{WB} \cdot ray\_body$                            # body to world frame
9: IF  $|ray\_world.z| < \epsilon$  THEN → RETURN None                # ray is parallel to the ground
10: t ← -position.z / ray_world.z
11: IF  $t < 0$  THEN → RETURN None                                # intersection behind the camera
12: p_world ← position + t · ray_world
13: RETURN p_world
```

3.3 Clustering & Estimation

We know that there are 6 targets with the dataset, which are present in multiple frames. In order to have a more accurate estimate for the targets' world positions, we cluster the raw position detections and reject any outlier clusters that might appear. For clustering and outlier rejection, we used the DBSCAN strategy from the `sklearn.cluster` library. DBSCAN groups points that are within a distance threshold (ϵ) of each other into clusters, and automatically identifies outliers as points that don't belong to any dense neighborhood.

FUNCTION EstimateTargets(D, ϵ):

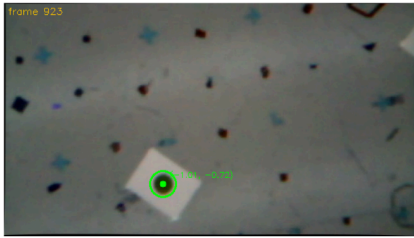
```
1: positions ← Extract (x, y) world coordinates from detections in D
2: labels ← DBSCAN(positions, eps =  $\epsilon$ , min_samples = 1)
3: clusters ← Group detections in D by labels
4: FOR each cluster  $C \in$  clusters DO
5:   mean_pos ← Mean(positions) ← Extract world positions from detections in C
6:   std_pos ← StandardDeviation(pos)
7:   mean_radius ← Mean(radii) ← Extract pixel radii from detections in C
8:   targets.append (target ← { mean_pos, std_pos, num_detections , mean_radius })
9: RETURN targets
```

4.0 Target Estimation:

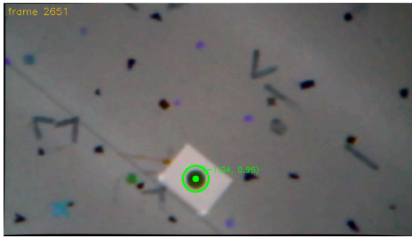
Target #	x position (m)	y position (m)	z position (m)	x_std	y_std	z_std	Mean radius	# of detections
1	-1.02058207	0.97068107	0.0	0.02068112	0.01300876	0.0	0.0196	256
2	-1.02555904	-0.70648038	0.0	0.02156794	0.02100099	0.0	0.021	252
3	0.05090615	-1.92976606	0.0	0.00783307	0.01376831	0.0	0.0196	44
4	0.3566132	-0.30913933	0.0	0.04036622	0.03110513	0.0	0.0196	96
5	0.63799617	0.85731404	0.0	0.06078027	0.01954592	0.0	0.0196	72
6	1.42361948	-1.47241559	0.0	0.01867231	0.02906234	0.0	0.0196	44

Representative Frame per Target

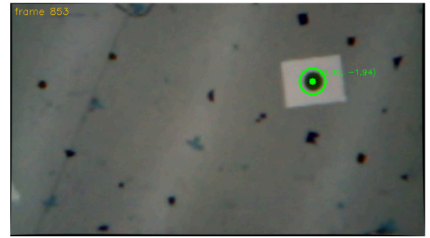
Target 2 (-1.023, -0.710)
220 detections



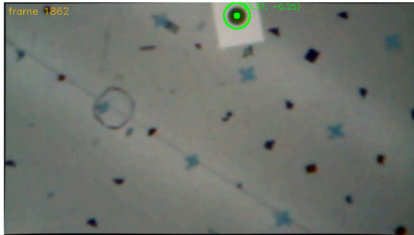
Target 1 (-1.023, 0.967)
280 detections



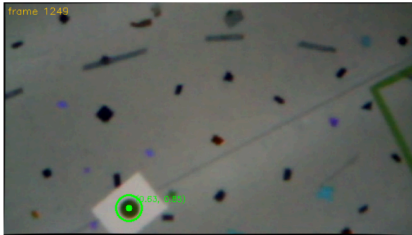
Target 3 (0.046, -1.939)
28 detections



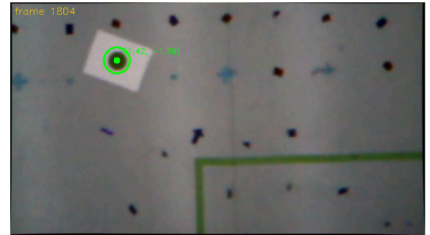
Target 5 (0.315, -0.259)
428 detections



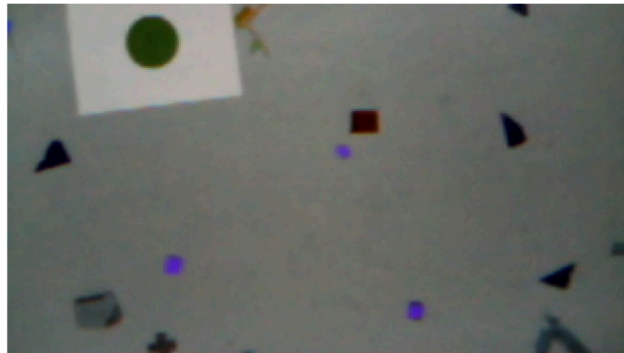
Target 4 (0.625, 0.860)
58 detections



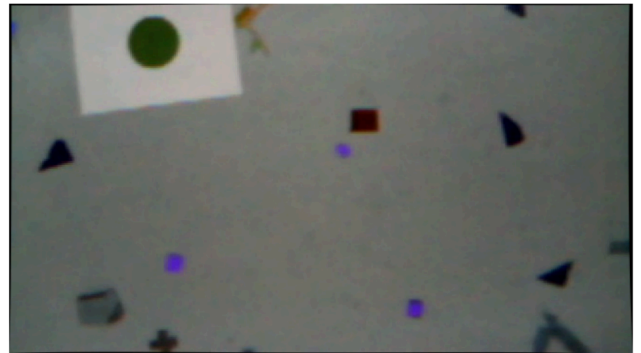
Target 6 (1.431, -1.483)
64 detections



Raw Image (frame 298)

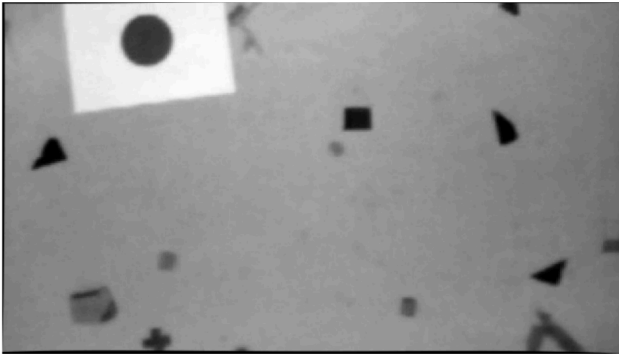


Undistorted Image (frame 298)



Contour Detection Pipeline (frame 298)

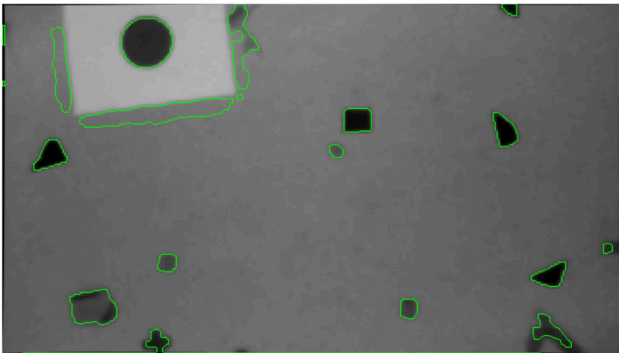
1. Grayscale



2. Adaptive Threshold (block=51, C=10)
+ Morphology (5x5 close/open)

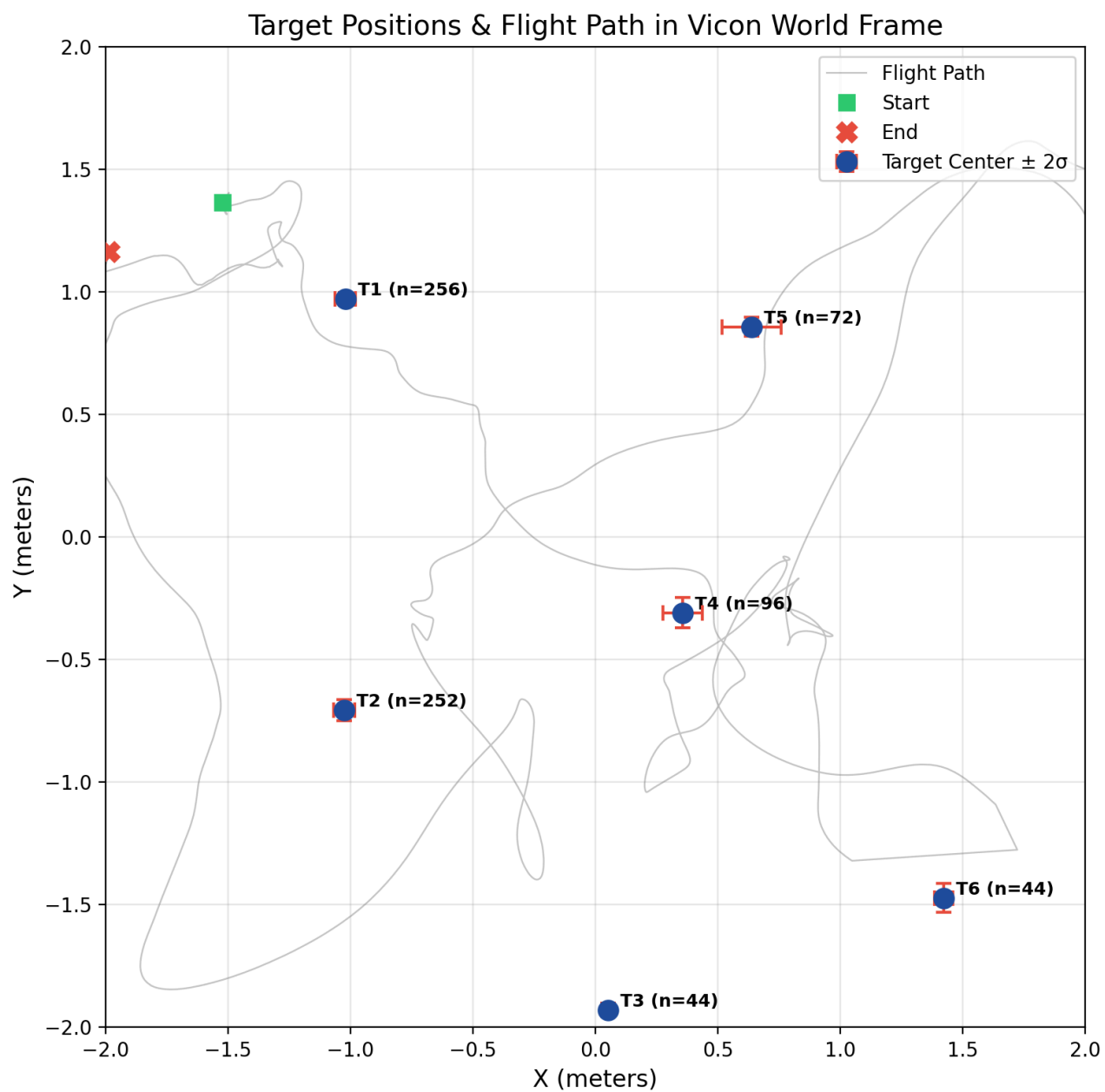


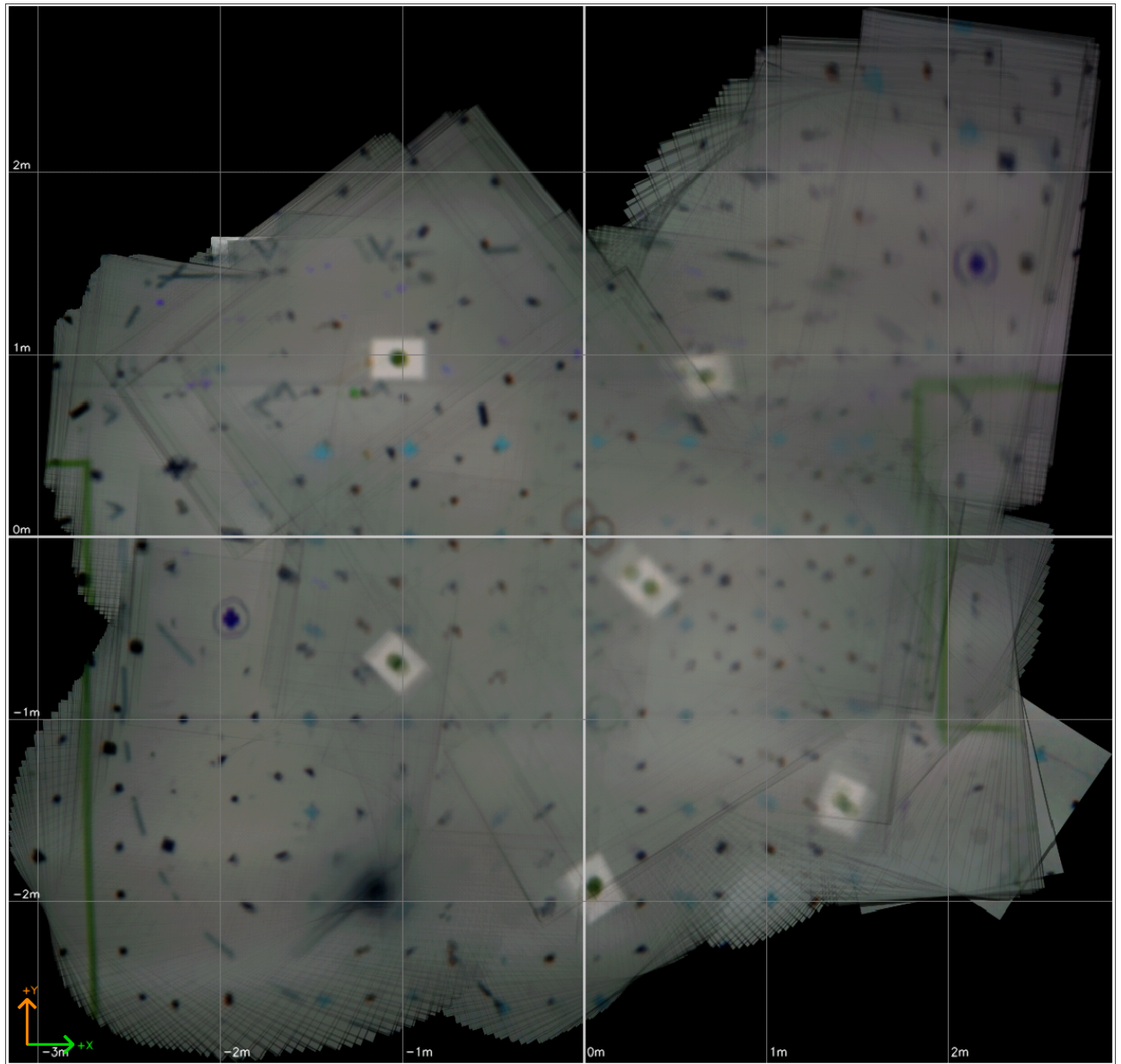
3. External Contours (20 found)



4. Filtered Detections (1 accepted, 15 rejected)







AI Acknowledgment

This project benefited from AI-assisted code organization and design critique. Claude was used to help clean up the structure of the codebase and provide feedback on architectural decisions, while all core ideas and implementation remain the work of the group members.

Appendix:

$$K = \begin{bmatrix} 698.86 & 0.0 & 306.91 \\ 0.0 & 699.13 & 150.34 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}$$

$$T_{CB} = \begin{bmatrix} 0.0 & -1.0 & 0.0 & 0.0 \\ -1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & -1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$